

the data into a bar chart:

```
print(df["Sentiment"].value_counts())
sns.countplot(x="Sentiment", data=df);
```

Figure 1 shows the distribution of positive, negative and neutral sentences in the data set.

Step 4: Next, we will start with cleaning and pre-processing the text before applying the MNB algorithm to boost the accuracy and precision of the training data:

```
stop_words = set(stopwords.words("english"))
df["Sentence"] = df["Sentence"].str.replace("\d", "")
def cleaner(data):

    # Tokens
    tokens = word_tokenize(str(data).replace("'", "").lower())

    # Remove Puncts
    without_punc = [w for w in tokens if w.isalpha()]

    # Stopwords
    without_sw = [t for t in without_punc if t not in stop_words]

    # Lemmatize
    text_len = [WordNetLemmatizer().lemmatize(t) for t in
without_sw]

    # Stem
    text_cleaned = [PorterStemmer().stem(w) for w in text_len]

    return " ".join(text_cleaned)

df["Sentence"] = df["Sentence"].apply(cleaner)
```

Step 5: Now we need to filter out the rare words that are used in the sentences.

```

rare_words = pd.Series(" ".join(df["Sentence"]).split()).
value_counts()
rare_words = rare_words[rare_words <= 2]

df["Sentence"] = df["Sentence"].apply(lambda x: " ".join([i
for i in x.split() if i not in rare_words.index]))

```

Step 6: As a next step, we can generate a word cloud to see the most commonly occurring words in the text. This is an option step and more for our own understanding of the data set than anything else. Figure 2 shows the word cloud generated as a result of the code in Step 6.

```
plt.figure(figsize=(16,12))
wordcloud = WordCloud(background_color="black",max_words=500,
width=1500, height=1000).generate(' '.join(df['Sentence']))
```

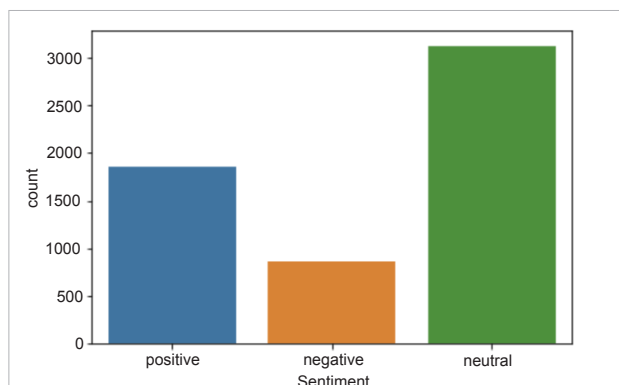


Figure 1: Distribution of positive, negative and neutral sentences in the data set



Figure 2: Word cloud showing 500 most commonly used terms in the data set

```
plt.imshow(wordcloud, interpolation='bilinear')
plt.axis("off")
plt.show()
```

Step 7: Before we execute the algorithm, we need to divide the data set into two parts. One part will be used for training the algorithm and the other for testing it.

```
from sklearn.model_selection import train_test_split
from sklearn.metrics import confusion_matrix, classification_
report, accuracy_score
from sklearn.naive_bayes import MultinomialNB
```

```
X = df["Sentence"]
y = df["Target"]
```

```
X_train,X_test,y_train,y_test = train_test_split(X,y, test_
size = 0.25,random_state= 42,stratify=y)
```

Step 8: Finally, we will execute the Multinomial Naïve Bayes algorithm on the training data:

```
nb_model = MultinomialNB()  
nb_model.fit(X_train_count,y_train)
```